

Тестовое задание. Backend-разработчик (Go)

Wildberries, Поиск

Проблема

На маркетплейсе каждую секунду пользователи совершают огромное количество поисковых запросов. Для улучшения пользовательского опыта мы планируем добавить на главную страницу виджет **«Сейчас ищут»**, который должен в реальном времени показывать актуальные тренды поиска.

Постановка задачи от команды продукта

«Нам нужен сервис, который будет отдавать Топ самых популярных поисковых запросов за последние 5 минут.

Поток сырых логов поиска будет непрерывно поступать к вам из другого сервиса. Виджет будет отображаться на главной странице у всех пользователей, поэтому ручка получения топа должна работать максимально быстро и выдерживать пиковые нагрузки.

Дополнительно маркетинг просит предусмотреть возможность скрывания нежелательных слов из топа (стоп-лист). А аналитики недавно жаловались, что иногда конкуренты или парсеры генерируют аномальные всплески одних и тех же запросов, что ломает нашу статистику — хотелось бы, чтобы на виджете эти накрутки не отражались.»

Требования и детали по заданию

Технические требования:

- **Язык разработки:** Go (Golang).
- **Взаимодействие:** Чтение входящих поисковых событий должно осуществляться через подписку на брокер сообщений (Kafka, NATS, RabbitMQ — на ваш выбор). Данные туда пишет смежный сервис поиска.
- **API:** Сервис должен предоставлять API (HTTP/JSON или gRPC) для взаимодействия с клиентами.
- **Инфраструктура:** Использование managed-решений от облачных провайдеров запрещено. стек технологий, базу данных (если она требуется) и структуры хранения данных вы выбираете самостоятельно, опираясь на ожидаемую нагрузку.
- **Нагрузка:** Сервис работает в условиях highload. Ожидается, что запросов на чтение топа будет на порядок больше (в 10-50 раз), чем входящих событий в брокере.

Основное задание (минимум):

- Консьюмер, читающий поток поисковых запросов из брокера сообщений.
- Метод получения актуального Топ-N запросов за последние 5 минут (где лимит N передается параметром).

Детали реализации:

- По умолчанию сервис не содержит никаких данных.
- В постановке задачи намеренно присутствуют неточности. Механизмы очистки данных и архитектурные подходы остаются полностью на ваше усмотрение.
- **Проектирование контракта:** Формат payload в брокере не задан. Представьте, что вы формируете контракт для смежной команды. Вы должны сами определить, какие именно данные вам нужны в каждом сообщении для выполнения всех бизнес-требований, и зафиксировать этот контракт в документации.

Будет плюсом (опционально)

Выполнение этих пунктов не обязательно, но даст существенный плюс:

- **Динамический стоп-лист:** Реализация API для управления стоп-листом (добавление и удаление нежелательных слов «на лету» без перезапуска сервиса).
- **Производительность:** Проведение нагрузочного тестирования и предоставление результатов бенчмарков (встроенными средствами Go или утилитами `wrk`, `hey`, `vegeta`), доказывающих, что ваша архитектура выдерживает высокую конкурентную нагрузку.
- **Мониторинг:** Добавление базовых метрик (например, в формате Prometheus) для наблюдения за состоянием сервиса.
- **Тестирование:** Покрытие ключевой бизнес-логики Unit-тестами.
- **DX:** Настроенный процесс быстрого локального запуска для проверяющих (например, `docker-compose` файл, поднимающий сам сервис и брокер сообщений).

Требования к передаче результатов

- Код должен быть выложен в публичный git-репозиторий (GitHub/GitLab).
- Проект **обязательно** должен содержать файл `README.md`.

Что мы ожидаем увидеть в README:

1. Инструкцию по локальному запуску проекта и примеры запросов к API.
2. **Контракт данных:** пример payload'a из брокера и обоснование, почему вам нужны именно эти поля.
3. **Обоснование архитектуры:** почему были выбраны именно такие структуры данных для хранения топа (и управления стоп-листом, если он реализован).
4. **Trade-offs и бизнес-логика:** описание компромиссов, на которые пришлось пойти ради производительности. Список проблем, которые вы выявили в продуктовой постановке задачи, и то, как вы их решили.